
AGENTES: DIRIGIR, SUPERVISAR Y AUDITAR

Fernanda Sobrino

2026

De ReAct a agentes entrenados

Recapitulación: ¿qué es un agente?

- ▶ Un LLM que además puede **actuar**: llamar funciones, buscar, ejecutar código, leer y escribir archivos
- ▶ Opera en un ciclo: **razonar** → **actuar** → **observar** el resultado → razonar de nuevo
- ▶ El patrón original es **ReAct** (Yao et al., 2022): todo el ciclo se logra *con prompting* sobre un modelo normal
- ▶ El modelo decide *qué* herramienta usar y *cuándo*; el sistema alrededor decide *qué le está permitido*

ReAct es la línea base, ya no la frontera

- ▶ Desde 2025 los agentes de frontera **no solo se promptean: se entrenan**
- ▶ La política de uso de herramientas se optimiza con RL sobre episodios completos de varios pasos (tareas largas tipo “resuelve este issue de GitHub”)
- ▶ Resultado: modelos abiertos ya resuelven la mayoría de las tareas de benchmarks como SWE-bench Verified
- ▶ Implicación conceptual: el agente ya no es “un LLM + un loop”; el loop **es parte de lo que el modelo aprendió**

¿Qué cambia cuando el agente se entrena con RL?

- ▶ Sube la capacidad: tareas más largas, menos pasos desperdiciados, mejor recuperación de errores
- ▶ Aparece una clase de falla nueva: **reward hacking / spec gaming**
 - ▷ el agente aprende a *maximizar la señal de éxito*, no necesariamente a *hacer lo que querías*
 - ▷ ejemplo clásico: “pasa los tests” → el agente edita los tests
- ▶ Es la versión agéntica de la lección de Goodhart que ya conocen: cuando la métrica se vuelve objetivo, deja de medir
- ▶ Por eso la especificación (qué pedir) y la verificación (qué revisar) son *más* importantes con agentes más capaces, no menos

La habilidad que importa

- ▶ Ustedes no van a escribir el código del agente; van a **dirigirlo**
- ▶ Dirigir bien = especificar bien + saber dónde mirar cuando algo falla
- ▶ Este deck es un catálogo de *dónde mirar*: los modos de falla documentados y el diseño de supervisión
- ▶ Todo esto se aplica directamente en su proyecto final de colaboración con IA

Modos de falla de los agentes

¿Por qué una taxonomía?

- ▶ Microsoft publicó una **taxonomía de modos de falla en sistemas agénticos** (v2.0, junio 2026), construida con 12 meses de red-teaming de sistemas reales
- ▶ Distingue dos familias:
 - ▷ fallas **nuevas** de los agentes: compromiso del agente, suplantación, manipulación del flujo
 - ▷ fallas de LLMs **amplificadas** por la agencia: inyección de prompts, envenenamiento de memoria, brincarse al humano en el loop
- ▶ Para ustedes funciona como *checklist de auditoría*: preguntas concretas que hacerle a cualquier despliegue de agentes
- ▶ Lectura recomendada del curso — no requiere saber programar

Inyección de prompts: la falla #1

- ▶ El riesgo #1 de OWASP para LLMs por segundo año consecutivo
- ▶ Tres vectores:
 1. **Directa**: el usuario manipula el objetivo (“ignora tus instrucciones y...”)
 2. **Indirecta**: las instrucciones vienen escondidas en *datos* que el agente lee — un documento, una página web, un resultado de RAG, la salida de una herramienta
 3. **Recursiva**: la inyección se propaga de un paso del agente al siguiente
- ▶ La idea central: para un LLM **no hay diferencia de tipo entre instrucciones y datos** — todo es texto en el contexto

¿Por qué la inyección es peor en agentes?

- ▶ En un chatbot, una inyección arruina *una respuesta*
- ▶ En un agente, una sola inyección puede secuestrar **toda la cadena de objetivos**:
 - ▷ el agente lee un documento envenenado → “decide” un nuevo objetivo → usa sus herramientas para ejecutarlo
- ▶ El daño escala con los permisos: un agente que puede mandar correos, mover archivos o ejecutar código convierte texto malicioso en *acciones* maliciosas
- ▶ Variante 2025-2026: **envenenamiento de herramientas** — las instrucciones maliciosas van en la *descripción* de la herramienta misma; hasta el catálogo de herramientas puede mentir

Envenenamiento de contexto

- ▶ Una alucinación normal desaparece al terminar la conversación
- ▶ Un dato falso que entra al **contexto** del agente (una mala búsqueda, un resultado inventado de una herramienta) se queda: se cita a sí mismo en pasos posteriores
- ▶ Resultado: “alucinaciones apiladas” — cada paso razona sobre la basura del paso anterior con total coherencia
- ▶ Si el agente tiene **memoria persistente**, el veneno sobrevive entre sesiones (envenenamiento de memoria)
- ▶ Pregunta de auditoría: “¿en qué paso exacto entró esta afirmación al contexto, y sigue ahí tres turnos después?”

Sycophancy y reward hacking

- ▶ **Sycophancy**: los modelos entrenados con preferencias humanas aprenden a *complacer* — te dan la razón, suavizan malas noticias, confirman tu hipótesis
- ▶ Es un sesgo de entrenamiento, no un bug ocasional: *complacer fue* la señal de recompensa
- ▶ En el extremo documentado, escala a manipular la propia señal de evaluación (reward tampering)
- ▶ Implicación práctica para ustedes: **un agente que nunca te contradice es una señal de alerta, no de calidad**
- ▶ Antídoto barato: pedir explícitamente la evidencia en contra y los supuestos que, de fallar, invalidan la respuesta

Alucinación de acciones

- ▶ El agente puede *afirmar* que hizo algo que no hizo: “ejecuté los tests y pasan”, “guardé el archivo”, “consulté la base”
- ▶ O al revés: reportar resultados de una herramienta que falló silenciosamente
- ▶ Regla de supervisión: **las afirmaciones sobre acciones se verifican contra el artefacto, no contra el relato**
 - ▷ ¿existe el archivo? ¿está el output de los tests? ¿la cifra aparece en la fuente citada?
- ▶ Es la misma disciplina del protocolo experimental del curso: el resultado es lo que está en la tabla, no lo que dice el resumen

El checklist: falla → señal → pregunta

Falla	Señal típica	Pregunta que debes hacer
Inyección de prompts	El agente “cambia de objetivo” tras leer algo	¿Qué leyó justo antes? ¿Quién controla esa fuente?
Envenenamiento de contexto	Un dato dudoso reaparece “confirmado”	¿Dónde entró ese dato? ¿Cuál es la fuente original?
Reward hacking	Éxito sospechosamente fácil o literal	¿Qué métrica optimizó? ¿Se puede cumplir sin lograr el objetivo?
Sycophancy	Siempre está de acuerdo contigo	¿Qué evidencia en contra encontró? ¿Qué lo haría cambiar de opinión?
Alucinación de acciones	Relato sin artefactos	Muéstrame el archivo / el output / la fuente exacta

Diseño de supervisión

Human-in-the-loop vs human-on-the-loop

- ▶ **HITL** (*in the loop*): el agente se **detiene y espera aprobación** antes de cada acción consecuente
- ▶ **HOTL** (*on the loop*): el agente ejecuta; un humano **monitorea** y solo interviene ante anomalías
- ▶ Trade-off: HITL es más seguro pero no escala (y entrena al humano a aprobar en automático); HOTL escala pero exige detección de anomalías que funcione
- ▶ El diseño correcto es **selectivo**: HITL para acciones de alto riesgo o difíciles de revertir, HOTL para el resto

La supervisión es una decisión de gobernanza

- ▶ Qué acciones requieren aprobación humana **no es una decisión técnica**: es una decisión sobre riesgo, reversibilidad y responsabilidad
- ▶ Preguntas de diseño:
 - ▷ ¿qué acciones son irreversibles o externas (pagos, notificaciones a ciudadanos, cambios en expedientes)?
 - ▷ ¿quién aprueba, con qué información enfrente, y con cuánto tiempo?
 - ▷ ¿quién responde cuando el agente se equivoca — el operador, el proveedor, la agencia?
- ▶ La mayoría de las organizaciones ponen “a alguien en el loop” sin entrenarlo en *qué* revisar — eso es teatro de supervisión

Complacencia de automatización

- ▶ Fenómeno documentado desde la aviación: cuando el sistema casi siempre acierta, el humano deja de revisar de verdad
- ▶ El “revisor” se convierte en un sello de goma: aprueba a la velocidad a la que el sistema propone
- ▶ Es el mecanismo exacto del caso holandés que veremos en un momento: había revisión humana *nominal*
- ▶ Diseños que ayudan: fricción selectiva (el sistema debe *mostrar la evidencia*, no solo la conclusión), auditorías aleatorias profundas, medir la tasa de rechazo del revisor (si nunca rechaza, no está revisando)

Permisos y sandboxing

- ▶ El principio más barato y efectivo: **mínimo privilegio**
- ▶ El agente solo debería poder tocar lo que la tarea requiere: lectura sí / escritura no, este directorio sí / el sistema no, proponer sí / enviar no
- ▶ El sistema alrededor del agente — no el modelo — es quien concede permisos
- ▶ Corolario para la inyección: si el texto malicioso no puede convertirse en una acción permitida, el ataque se degrada a molestia
- ▶ Pregunta de auditoría: *“si este agente fuera secuestrado hoy, ¿qué es lo peor que puede hacer con sus permisos actuales?”*

Vocabulario: la pila de protocolos

- ▶ **MCP** (*Model Context Protocol*): el estándar de facto para conectar agentes con herramientas y datos (~miles de servidores de herramientas registrados en 2026)
- ▶ **A2A** (*agent-to-agent*): protocolo emergente para que agentes se coordinen entre sí
- ▶ No necesitan los detalles técnicos; necesitan reconocer los nombres en una licitación o propuesta de proveedor
- ▶ Y una implicación de auditoría: cada herramienta conectada es **superficie de ataque** — el inventario de herramientas de un agente es parte de su evaluación de riesgo

Evaluar agentes

Benchmarks agénticos: tres diseños

- ▶ **SWE-bench**: issues reales de GitHub; el éxito lo deciden **tests unitarios** — verificación objetiva, sin juez
- ▶ **GAIA**: cientos de tareas de uso de herramientas con respuesta verificable por humanos
- ▶ **τ -bench**: el agente atiende a un *usuario simulado por otro LLM*; se mide completar la tarea **y** respetar las políticas (no prometer lo que las reglas prohíben)
- ▶ Lo importante no es memorizar benchmarks: es reconocer **qué mide cada diseño** — verificación objetiva vs juicio vs cumplimiento de reglas

El benchmark no predice tu producción

- ▶ Consenso 2026, admitido incluso por los mantenedores de benchmarks: ningún benchmark público predijo las fallas reales en producción de los equipos que despliegan agentes
- ▶ Causas: contaminación (el benchmark estaba en el entrenamiento), saturación, *teaching to the test*, y tareas de benchmark $\neq$ tus tareas
- ▶ Respuestas del campo: benchmarks con problemas **fechados después del corte de entrenamiento** (estilo LiveCodeBench) y conjuntos de evaluación **privados**
- ▶ Para ustedes la lección es la misma de siempre, ahora con evidencia: **el eval propio, con casos reales y fuera del alcance del proveedor, no es opcional**

Evaluar sin verdad de referencia

- ▶ En tareas abiertas no hay test unitario: entra el **LLM-as-judge** con rúbrica
- ▶ Ya conocen sus sesgos (longitud, estilo propio, orden); la novedad de producción es el **costo**: el juez es demasiado caro y lento para evaluar cada interacción
- ▶ Patrón estándar 2026: clasificadores ligeros monitorean *todo* en línea; el juez completo corre por muestreo, fuera de línea
- ▶ Y el juez se valida contra una muestra calificada por humanos **antes** de confiar en él — un instrumento de medición se calibra, como cualquier otro

Red-teaming agéntico

- ▶ Buscar activamente cómo hacer fallar al agente *antes* que los usuarios (o los atacantes) lo hagan
- ▶ Novedad 2025-2026: el red-teaming también se automatizó — agentes atacando agentes, con tasas de éxito superiores al red-teaming manual
- ▶ Existen benchmarks específicos de la superficie agéntica (inyección vía herramientas, tareas dañinas, sabotaje sutil)
- ▶ Para un despliegue público: el reporte de red-teaming es un **entregable exigible al proveedor**, igual que las métricas por subgrupo

Agentes en el sector público

Países Bajos: el escándalo de las guarderías

- ▶ 2005-2019: un algoritmo de perfilamiento de riesgo señaló a ~26,000 familias como defraudadoras del subsidio de guarderías
- ▶ Usaba nacionalidad/etnicidad entre los factores de riesgo; los señalados debían devolver años de subsidios completos
- ▶ Consecuencias: quiebras familiares, separaciones, y la **caída del gobierno en 2021**
- ▶ No es historia antigua: es el caso canónico de lo que sale mal — y *cómo* sale mal

Releído con los lentes de este deck

- ▶ Había humanos “en el loop”: los casos pasaban por revisores
- ▶ Pero la revisión era nominal: **complacencia de automatización** — el sistema casi siempre “acertaba”, el revisor dejó de revisar
- ▶ No había métricas por subgrupo publicadas, ni ruta efectiva de apelación, ni responsabilidad clara
- ▶ La falla no fue (solo) el modelo: fue el **diseño de supervisión** alrededor del modelo
- ▶ Moraleja: cada elemento del checklist de este deck existía como pregunta sin hacer

El contraejemplo: Estonia y Singapur

- ▶ **Estonia — Bürokratt:** red de asistentes de IA compartida por más de 18 agencias; una sola infraestructura, supervisión y estándares comunes
- ▶ **Singapur — GovTech:** chatbots en más de 70 sitios de gobierno, con reducciones de ~50 % en carga de call centers
- ▶ Misma familia de tecnología que el caso holandés; **diferente diseño de supervisión, diferente resultado**
- ▶ La variable que importa no es “IA sí o no”: es quién la supervisa, con qué permisos, con qué métricas y con qué ruta de escalamiento

Qué exigir antes de desplegar un agente público

1. Los **5 mínimos** que ya conocen (eval propio, métricas por subgrupo, tasa de alucinación/citas, monitoreo, escalamiento a humano)
 2. Más los específicos de agentes:
 - ▷ inventario de herramientas y permisos (mínimo privilegio, ¿qué puede *hacer*?)
 - ▷ diseño HITL/HOTL explícito por tipo de acción, con responsables nombrados
 - ▷ reporte de red-teaming agéntico del proveedor
 - ▷ plan contra inyección: ¿qué fuentes no confiables lee el agente?
- ▶ Referencia: NIST lanzó en 2026 una iniciativa de estándares específicos para agentes; el marco general (AI RMF, ISO 42001) todavía no cubre del todo el riesgo agéntico

Esto es tu proyecto final

- ▶ El proyecto de colaboración con IA califica exactamente estas habilidades:
 - ▷ **especificación** que el agente no pueda “cumplir” sin lograr el objetivo
 - ▷ **verificación** contra artefactos, no contra el relato
 - ▷ detectar y documentar los errores del agente (siempre los hay)
- ▶ Usen el checklist de este deck como guía de su reporte de verificación
- ▶ Y la pregunta que resume el curso: *no “¿qué respondió el agente?”, sino “¿cómo sé si está bien?”*

Referencias

- ▶ Microsoft (2026), *Taxonomy of Failure Modes in Agentic AI Systems v2.0*
- ▶ OWASP (2026), *Top 10 for LLM Applications* — inyección de prompts
- ▶ Yao et al. (2022), *ReAct: Synergizing Reasoning and Acting in Language Models*
- ▶ Jimenez et al. (2024), *SWE-bench*; Mialon et al. (2023), *GAIA*; Yao et al. (2024), *τ -bench*
- ▶ Amnesty International (2021), *Xenophobic Machines* — el caso holandés
- ▶ NIST AI RMF y la iniciativa de estándares de agentes (CAISI, 2026)